Technical Requirement Document (TRD)

System Documentation | IRVision AI

1. Architecture Overview

IRVision AI is structured as a decoupled Single Page Application (SPA) utilizing a FastAPI backend service and a Vite + React frontend client:

- **Frontend Stack:** React 18, TypeScript, Tailwind CSS, Axios, Framer Motion, React Icons.
- **Backend Stack:** FastAPI (Python 3.10+), Uvicorn, Motor (Async MongoDB Driver).
- **Database Wrapper:** Dynamic database wrapper class implementing local JSON serialization file fallback (local_db.json) if MongoDB cloud clusters are unreachable.

2. Image Processing Pipeline Flow

Step	Algorithm / Library	Input	Output / Progress
1. Load & Standardize	OpenCV IMREAD / Resize	Grayscale/RGB image	Normalized BGR (15%)
2. Noise Speckle Filter	OpenCV GaussianBlur	Normalised image	Denoised grayscale (30%)
3. Contrast Equalization	OpenCV CLAHE	Denoised grayscale	High-contrast luminance (45%)
4. Super Resolution	OpenCV Interpolation + Unsharp	Contrast image	Sharp 1.5x upscaled (65%)
5. Multispectral Color	OpenCV Colormap (JET) + Blend	Sharp grayscale	Pseudo-colorized RGB (80%)
6. Edge Preservation	OpenCV Bilateral Filter	Colorized RGB	Clean boundary image (95%)
7. Response Packaging	JSON Metadata Extraction	Final products	Response payload (100%)

3. Key API Endpoints

- **POST /api/auth/register:** Registers a new scientist account with hashed credentials.
- **POST /api/auth/login:** Authenticates and returns a signed 24h JWT token.
- **POST /api/auth/forgot-password:** Generates and emails a 6-digit OTP code.

- **POST /api/auth/reset-password:** Verifies the OTP and updates the user's password.
- **POST /api/upload/process:** Receives multipart form data file and streams processing progress using SSE content-type.
- **GET /api/history:** Lists user uploads metadata, processing times, and file sizes.